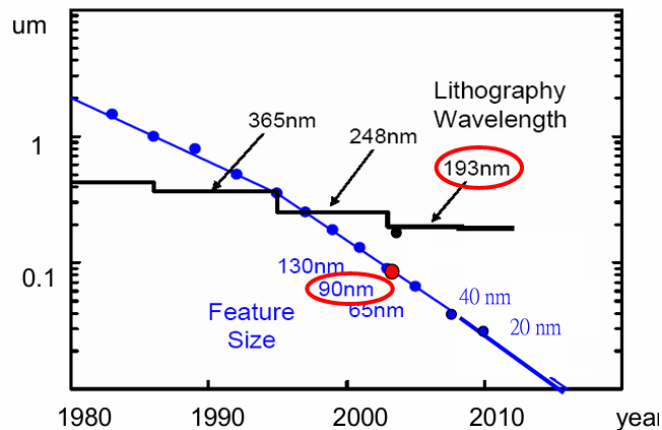
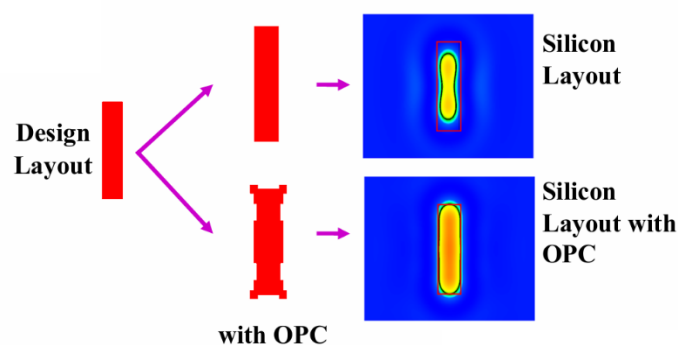


1. Explain how process limitations, such as lithography printability issues and process variation, impact chip manufacturing. How does Optical Proximity Correction (OPC) help address these challenges?



在 Lithography 的過程中，隨著製程技術的進步，我們需要印出更小的 Feature。如上圖可以看到，光的波長已經趕不上 Feature Size 的減小，因此光的繞射問題影響會更大，會導致更嚴重的 Pattern Distortion。

而 Process Variation 是指說，即使在相同的製造條件下，元件的尺寸或材料特性也會有些微的差異，或是製造設備的一些誤差，最後造成做出來的東西每次都不太一樣。像是在 Lithography 中，即使用相同的 Mask，印出來的 Pattern 也會有差。



先前提到，光的繞射問題會讓印出來的 Pattern 變形，導致結果不如預期，因此會需要使用 OPC 的技術，提前考慮光繞射之後會印出的結果，預先補足要使用的 Mask，這樣 Lithography 後就可以得到預期的 Pattern。

2. Refer to p.6 ~ p.30, unit 1. In the MANA algorithm, different partial paths can reach the same grid point with varying lengths. The first pruning strategy ensures that the difference between the min length of one partial path and the max length of another one remains within a specific bound d . Assume that the minimum required wire length is L .

a. What is the value of d ?

$$d = 2L-1$$

b. Provide a clear explanation of why this bound holds

已知 minimum required wire length 為 L ，根據 min length、max length 的定義，可以知道任何 Partial Path 的 $(\text{max length} - \text{min length}) \leq L$ 。

且套用 pruning strategy 1 後，可以確保某個 Partial Path 的 min length 一定小於另一個 Partial Path 的 max length。

假設存在兩條 Partial Path $P1$ 和 $P2$ ，且 $\text{maxlen}(P2) > \text{maxlen}(P1)$ ，最極端的情況是：

$$\text{maxlen}(P1) - \text{minlen}(P1) = L$$

$$\text{maxlen}(P2) - \text{minlen}(P2) = L$$

$$\text{且 } \text{maxlen}(P1) = \text{minlen}(P2) + 1$$

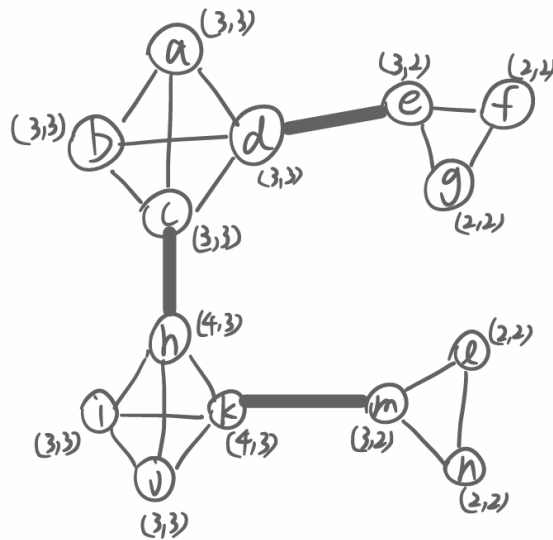
(這行是在符合 pruning strategy 1 的情況下最極端的結果)

因此

$$\begin{aligned} d &= \text{maxlen}(P2) - \text{minlen}(p1) \\ &= (\text{minlen}(P2) + L) - (\text{maxlen}(P1) - L) \\ &= (\text{maxlen}(P1) - 1 + L) - (\text{maxlen}(P1) - L) \\ &= 2L-1 \end{aligned}$$

3. Refer to p.8 ~ p.14, unit 2, answer the following questions.

a. Draw the conflict graph of the given design.



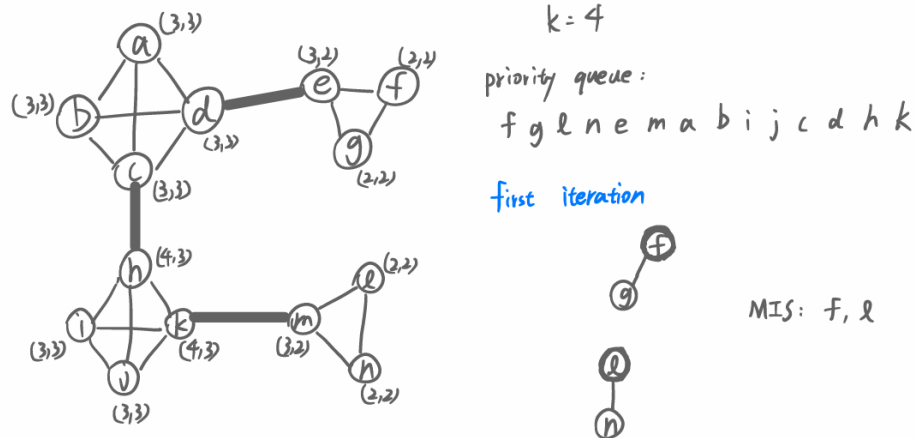
粗線代表是 External Edge

Node 的數字代表 (Degree, Feasible Number)

b. List the vertex degrees and feasible numbers of all the vertices.

Vertex	Degree	Feasible Number
a	3	3
b	3	3
c	4	3
d	4	3
e	3	2
f	2	2
g	2	2
h	4	3
i	3	3
j	3	3
k	4	3
l	2	2
m	3	2
n	2	2

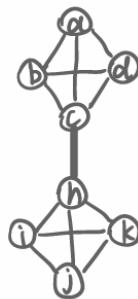
- c. List the vertices in the maximal independent set solution and draw the updated conflict graph after the first iteration of H2K. Assume the subgraph size $k = 4$, and assign higher priority to a vertex based on (1) a smaller degree, then (2) a smaller feasible number. If both (1) and (2) are the same, break ties with vertex names in alphabetical order (a, b, c, ...).



在第一個 Iteration 中，我選擇 {f, l}

因此新的 conflict graph 要去掉 {f, l, e, g, m, n}

updated conflict graph



- d. Propose a heuristic in addition to degree and feasible numbers that could be used to improve the subgraph extraction process in H2K. Explain how the proposed heuristic would be computed and incorporated into the priority queue of H2K.

在 *Post-Routing Redundant Via Insertion for Yield/Reliability Improvement* 裡面，作者提到可以藉由 redundant via 是否為 On-track 來當作 priority queue 的第三個 key (H3K)。

On-track redundant via 是指該 redundant via 是被 insert 到已經存在的 wire segment 上 (連接到 single via 的 wire)，與 Off-track 的 redundant via 相

比，On-track 會花比較少的資源且有更好的電性，因此更 preferable。

當 priority-queue 前兩個 key (Degree、Feasible Number) 都相同時，可以藉由是否為 On-track 來進一步決定順序。

4. Refer to p.14 ~ p.18, unit 3, answer the following questions.

a. How does Iterated Monte-Carlo (IMC) optimize the number of inserted fills?

IMC 會交替使用 Min-Var Objective 和 Min-Fill Objective 的演算法來優化 fill 的數量，根據講義的第 16 頁，大致可分為三步：

1. 執行 Min-Var Monte-Carlo with maximum window density U
2. 如果 minimum window density 沒有變，就結束
3. 執行 Min-Fill Monte-Carlo Algorithm with minimum window density M

這樣同時兼顧了 window fill density 和 fill 的總數

b. Compare Iterated Monte-Carlo (IMC) and Monte-Carlo (MC) by discussing the trade-offs in the following aspects. For each aspect, briefly explain why IMC behaves differently from MC.

i. Computational time

Monte-Carlo 專注在處理 Min-Var 的 Objective，只需要在乎 tile 被選中的機率（根據包含該 tile 的 window 中的 minimum density 值），不需要考慮其他目標，因此速度很快。

Iterated Monte-Carlo 則是藉由多次迭代，交替執行 Min-Var Objective 和 Min-Fill Objective 的演算法，犧牲一些執行速度換取更好的結果（更貼近最佳解），因此速度會比 Monte-Carlo 慢。

ii. Density variation control

如上述提到，Monte-Carlo 只考慮選擇的機率，不考慮其他目標，所以沒有機制可以減少 Excessive fill，而 Iterated Monte-Carlo 採用更系統化的方法增加或減少 fill，因此在 Density variation control 上，Iterated Monte-Carlo 更厲害。